FIPU

Kako web aplikacije doista rade

Web aplikacije · Predavanje 01

izv. prof. dr. sc. Nikola Tanković
Fakultet informatike, Sveučilište Jurja Dobrile u Puli

FIPU

Primjer

Stranica se otvorila. Gdje su proizvodi?

Sučelje je vidljivo, ali umjesto proizvoda piše **„Učitavanje…”**.

Što smijemo zaključiti?

A Backend sigurno ne radi.
B Baza je prazna.
C Trebamo još dokaza.

Ishodi i predznanje

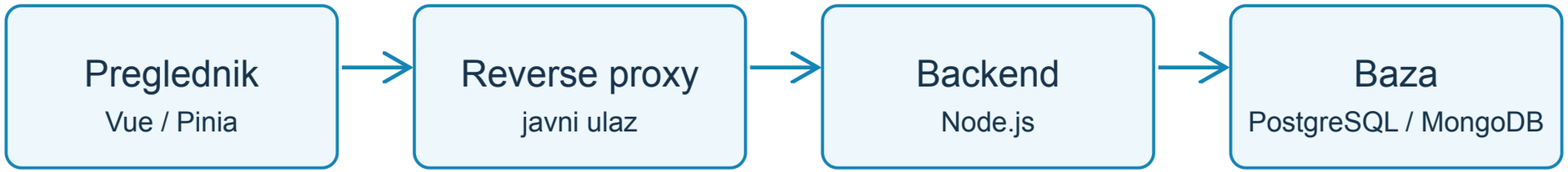
Nakon predavanja moći ćete:

- pratiti zahtjev od preglednika do baze i natrag
- odrediti odgovornosti komponenti i mjesta pohrane stanja
- razlikovati HTTP zahtjev, vezu i prijavljenu sesiju
- protumačiti poruku, status i format podataka
- predložiti provjeru mjesta kašnjenja ili ispada

Predznanje: JavaScript, osnovni HTTP, Vue 3 i Pinia.

Karta naše aplikacije

Logičke komponente, ne zadani broj računala



Strelice: zahtjev → ovisnost; odgovori se vraćaju obrnutim putem.

Preglednik nema izravan pristup bazi u ovom modelu.

Jedna stranica surađuje s više komponenti.

Preglednik prikazuje podatke; backend provodi pravila; baza čuva zapise.

Program, proces i poslužitelj

Pojam	U našem primjeru
Program	izvorni kod backenda
Proces	pokrenuta instanca Node.js-a
Poslužitelj	uloga procesa koji prima zahtjeve
Računalo	resurs na kojem procesi rade

URL nije identifikator jednog procesa.

HTML aplikacije i podaci proizvoda

1. Preglednik dohvaća HTML, JavaScript i stilove.
2. JavaScript pokreće `GET /products`.
3. Odgovor se pretvara u podatke za prikaz.

Vidljiv izbornik ne dokazuje uspješan dohvat proizvoda.

Granica povjerenja

Klijent šalje podatke koje može izmijeniti.

- Preglednik pomaže korisniku ispuniti zahtjev.
- Backend odlučuje što je dopušteno.
- Baza provodi definirana ograničenja podataka.

Skriven gumb ne sprječava izravan API zahtjev.

Primjer

Naivan dohvat proizvoda

```
1  const response = await fetch('/products');
2  products.value = await response.json();
```

Što prikazujemo tijekom čekanja, za `503` ili za `[]`?

Razlikuj stanja prikaza

Stanje	Poruka korisniku
Čekanje	Učitavanje proizvoda…
Uspjeh, postoje podaci	Popis proizvoda
Uspjeh, prazna lista	Nema proizvoda za ovaj upit
Neuspjeh	Dohvat nije uspio

Prazna lista nije zamjena za grešku.

Zahtjev koji pratimo

```
GET /products?limit=10 HTTP/1.1
Host: shop.example
Accept: application/json
```

Metoda • odredište • zaglavlja • moguće tijelo

Ovo je čitljiv zapis **HTTP/1.1**, ne prikaz bajtova HTTP/2 ili HTTP/3.

Adresa i ruta

`https://shop.example:443/products?limit=10#ponuda`

Dio	Uloga
`https`	shema
`shop.example:443`	naziv odredišta i port
`/products`	putanja
`limit=10`	parametar upita
`#ponuda`	fragment za klijenta; ne šalje se u zahtjevu

Metoda opisuje namjeru

Metoda	Namjera u HTTP-u
GET	dohvat reprezentacije
POST	obrada poslanog sadržaja
PUT	stvaranje ili zamjena stanja ciljanog resursa
PATCH	djelomična izmjena
DELETE	uklanjanje veze ciljanog resursa s funkcionalnošću

`GET /products` **ne smije naručivati proizvod**.

Odgovor: status, zaglavlja, tijelo

```
1 HTTP/1.1 200 OK
2 Content-Type: application/json
3
4 [{"id":7,"name":"Bilježnica","price":3.5}]
```

Status opisuje ishod zahtjeva. Tijelo nosi podatke.

Skupine statusa i granice zaključka

Status	Što provjeriti
2xx	uspjeh; zatim oblik podataka i prikaz
3xx	preusmjeravanje ili drugi oblik daljnjeg postupanja
4xx	problem zahtjeva ili pristupa resursu
5xx	poslužitelj nije ispunio zahtjev
Bez HTTP odgovora	mrežu, TLS, prekid ili blokiranje pristupa

Ne postoji HTTP status „0” koji je poslao server.

Accept i Content-Type

`Accept: application/json`

Što klijent prihvaća kao odgovor?

`Content-Type: application/json`

Koji format ima ovo tijelo poruke?

Zaglavlje **ne pretvara** HTML ili objekt u JSON.

Prije HTTP-a: DNS

`shop.example` → mrežna adresa

- Rezultat može već biti u predmemoriji.
- DNS ne bira handler `/products`.
- Neuspjeh razrješavanja može zaustaviti put prije API-ja.

Domena, IP adresa i ruta različite su stvari.

Veza i port

Za HTTP/1.1 i HTTP/2 tipično pratimo **TCP vezu**.

- IP i port vode do mrežnog odredišta.
- TCP uspostavlja vezu i prenosi tok bajtova.
- Aplikacija mora obraditi primljenu poruku.

HTTP/3 koristi QUIC nad UDP-om; današnji slijed modelira TCP put.

HTTPS i granica zaštite

TLS štiti komunikaciju između svojih krajnjih točaka.

Preglednik provjerava identitet poslužitelja putem certifikata.

TLS ne provjerava:

- smije li korisnik čitati tuđu narudžbu
- je li cijena u zahtjevu ispravna
- postoji li greška u backend kodu

Reverse proxy je posrednik

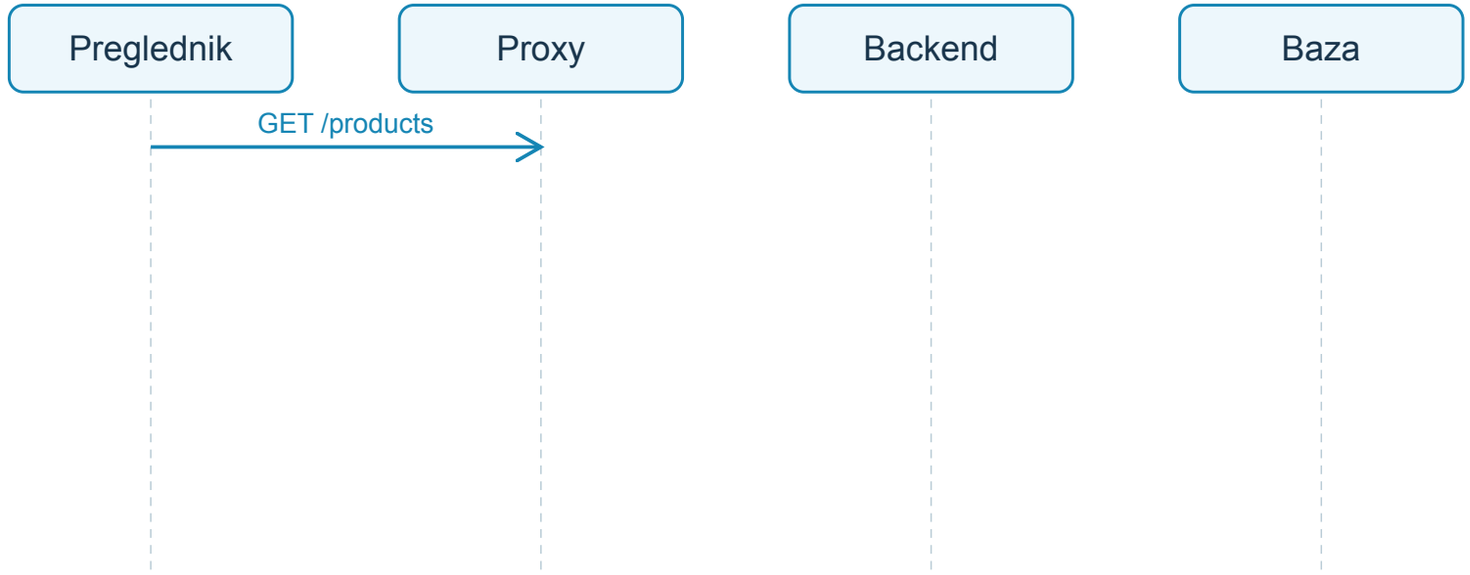
Javni ulaz nije nužno backend proces.

Proxy može:

- završavati TLS i prosljeđivati zahtjeve
- usmjeravati prema backend instanci
- vratiti vlastitu pogrešku ako backend ne odgovori

502/504 prvo povezujemo s komponentom koja ih je vratila.

Slijed 1: preglednik šalje zahtjev



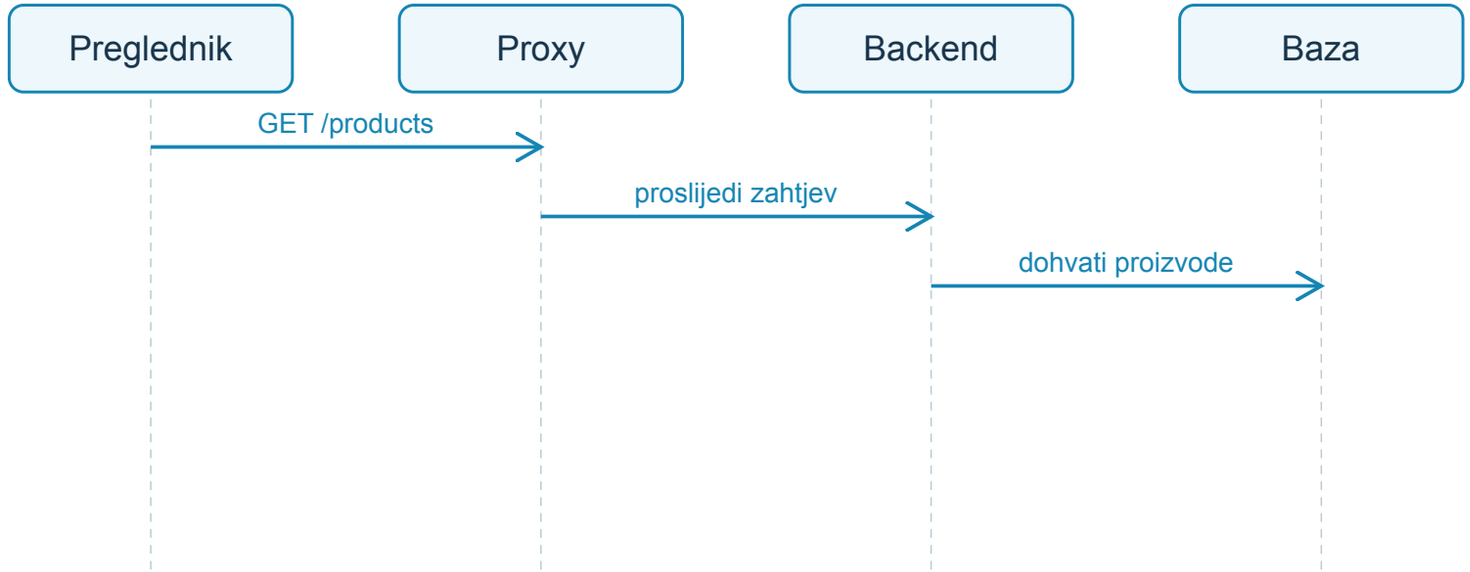
Vrijeme teče prema dolje. Pune strelice: pozivi; isprekidane: odgovori.

Vue akcija pokreće **`GET /products`**.

Prije slanja: `loading = true`.

Pretpostavke: veza je dostupna; zanemarujemo cache; jedan zahtjev.

Slijed 2: backend traži podatke



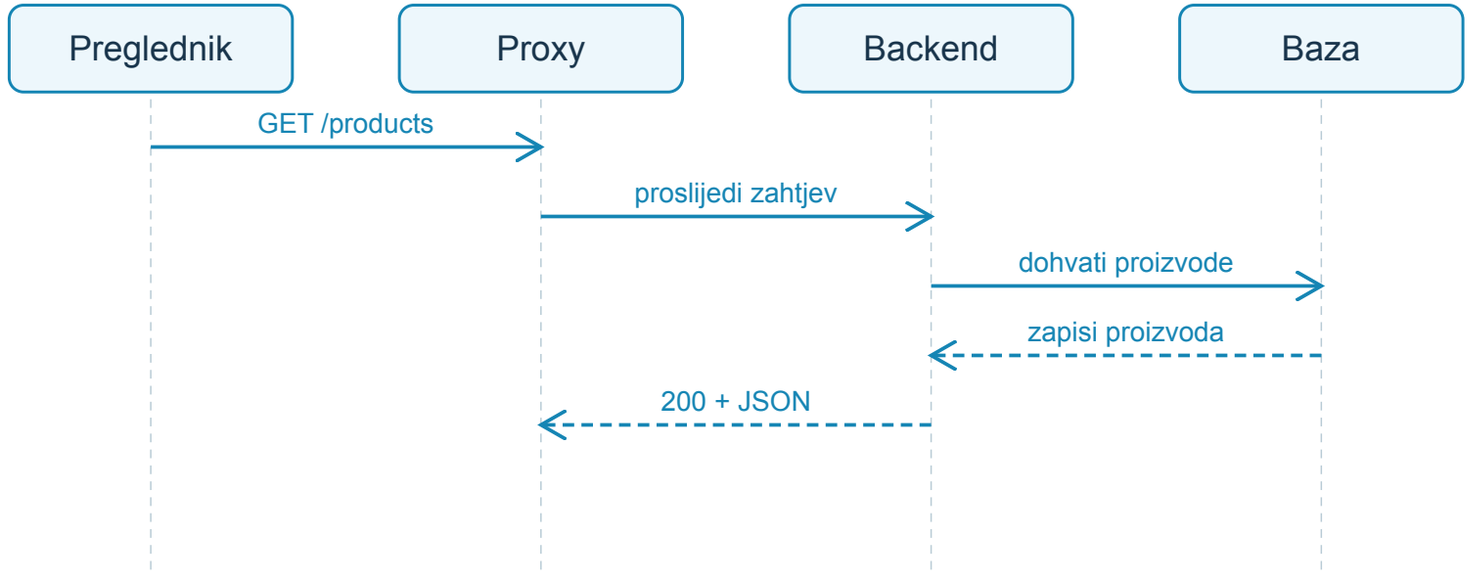
Vrijeme teče prema dolje. Pune strelice: pozivi; isprekidane: odgovori.

Proxy prosljeđuje poruku.

Backend odabire handler i traži proizvode iz baze.

Čekanje baze nije isto što i slanje odgovora.

Slijed 3: backend oblikuje odgovor



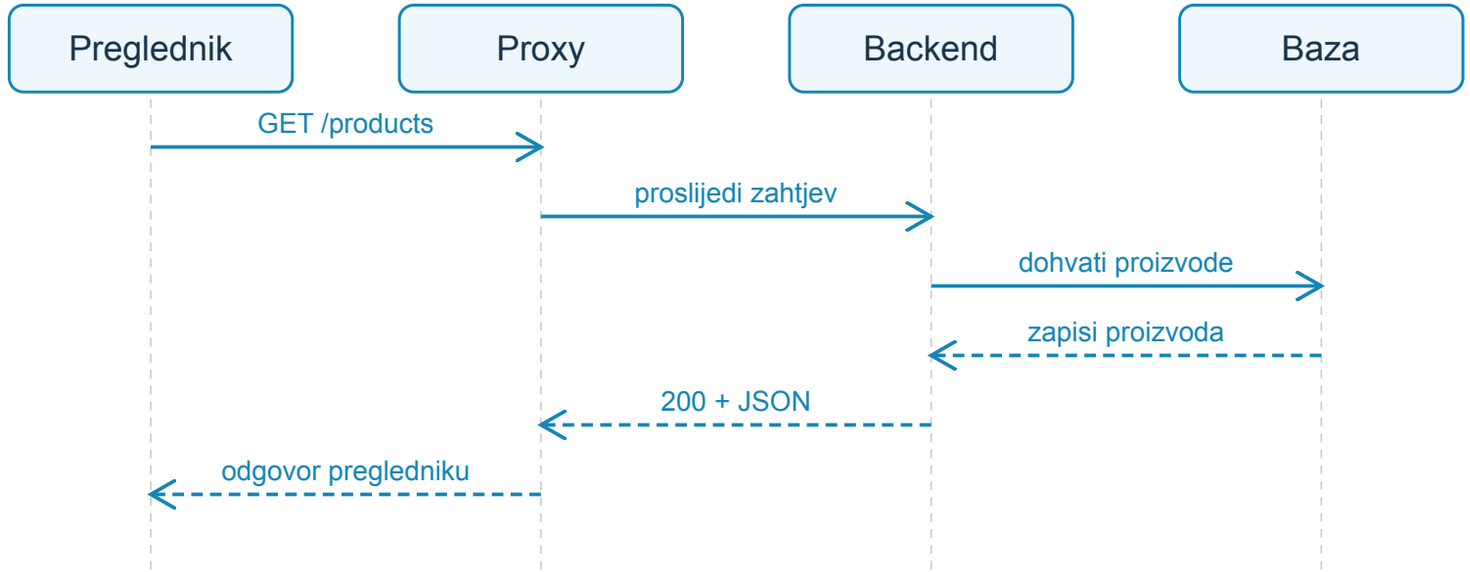
Vrijeme teče prema dolje. Pune strelice: pozivi; isprekidane: odgovori.

Baza vraća zapise.

Backend odabire javna polja i serijalizira odgovor.

Zapis baze i API reprezentacija ne moraju biti isti.

Slijed 4: prikaz se ažurira



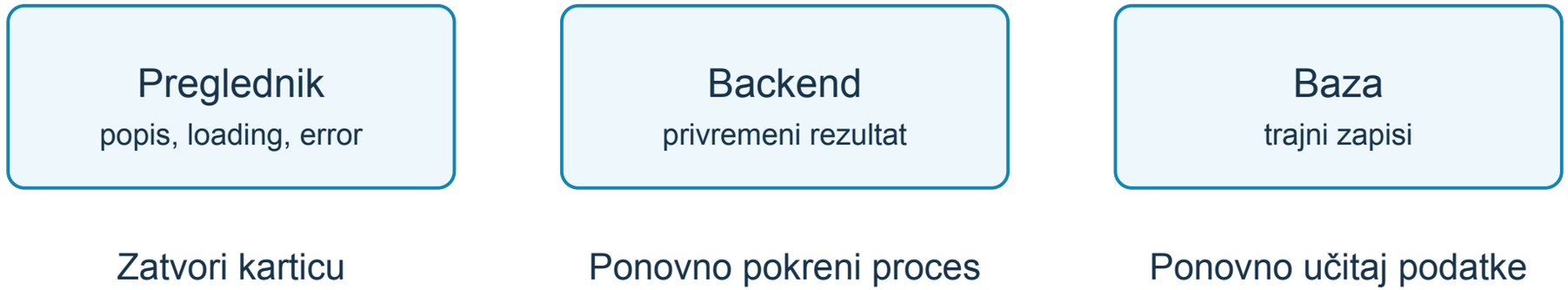
Vrijeme teče prema dolje. Pune strelice: pozivi; isprekidane: odgovori.

Preglednik prima status i tijelo.

Kod provjerava odgovor, čita JSON i ažurira stanje.

Tek tada komponenta može prikazati proizvode.

Gdje je stanje?



Pitaj: što nestaje, što ostaje i gdje postoji druga kopija?

Zatvaranje kartice, restart procesa i gubitak podataka nisu isto.

Za svaku vrijednost pitajte: tko je čuva, koliko dugo i gdje postoji druga kopija?

FIPU

Što znači HTTP stateless?

Definicija 1.1

HTTP je **stateless**: značenje zahtjeva ne oslanja se na obvezan prethodni dijalog koji HTTP pamti za tog klijenta.

Aplikacija i dalje može imati **korisnike, košarice i sesije**.

Trajna veza nije korisnička sesija



Jedna trajna veza može prenositi više zasebnih HTTP zahtjeva.

Veza prenosi poruke. Sesija povezuje zahtjeve s aplikacijskim kontekstom.

Jedan korisnik može koristiti više veza; jedna veza ne znači jednu poslovnu operaciju.

Cookie prenosi kontekst

```
Set-Cookie: sid=opaque-id; Secure; HttpOnly; SameSite=Lax
```

Kasniji prikladni zahtjev može sadržavati:

```
Cookie: sid=opaque-id
```

Cookie je mehanizam prijenosa; pravila sesije određuje aplikacija.

CORS: odgovor i pristup iz JavaScripta

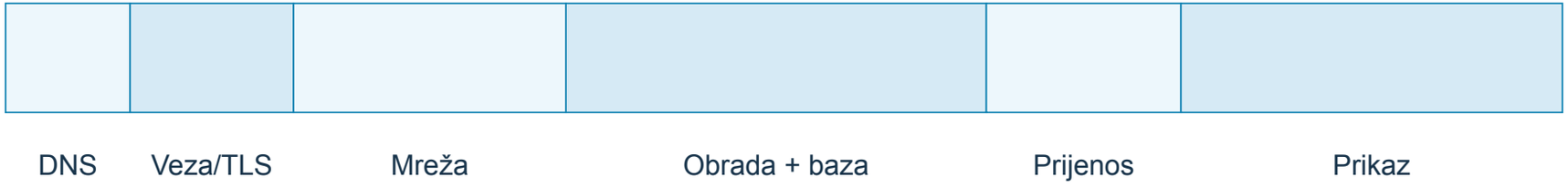
Podrijetlo čine **shema, naziv domaćina i port**.

`http://localhost:5173` ≠ `http://localhost:3000`

- Preglednik ograničava čitanje odgovora drugog podrijetla.
- CORS određuje dopušten pristup.
- CORS **nije autentikacija ni autorizacija API-ja**.

Gdje odlazi vrijeme?

Ilustrativna razdioba vremena — nije mjerenje



TTFB završava pri prvom bajtu odgovora; prikaz dolazi poslije.

Ponavljanje može preskočiti DNS i uspostavu već postojeće veze.

Vrijeme do prikaza obuhvaća više od rada handlera.

Mjerimo određenu granicu; ne nazivamo sve „sporim serverom”.

Bez odgovora: što se doista dogodilo?



Zahtjev može biti obrađen, a odgovor nečitljiv ili izgubljen.

Istek čekanja govori da nismo dobili rezultat na vrijeme.

Ne dokazuje da backend nije radio.

Dohvat s eksplicitnom provjerom

```
loading.value = true;
error.value = null;
try {
  const r = await fetch('/products');
  if (!r.ok) throw new Error(`HTTP ${r.status}`);
  const data = await r.json();
  if (!Array.isArray(data)) throw new Error('Oblik podataka');
  products.value = data;
} catch (e) {
  error.value = 'Dohvat nije uspio';
} finally {
  loading.value = false;
}
```

Skraćen primjer: provjera polja proizvoda slijedi iz API ugovora.

Handler je granica obrade

```
// Konceptualni kod; nije određeni framework API.
async function listProducts(request) {
  const limit = validateLimit(request.query.limit);
  const rows = await repository.findProducts(limit);
  return { status: 200, body: publicProducts(rows) };
}
```

Usmjeravanje, pravila, dohvat i odgovor imaju različite uloge.

Vježba: locirajte sljedeću provjeru

Opažanje	Gdje biste dalje gledali?
Nema `/products` u Networku	?
`200`, tijelo `[]`	?
`200`, proizvodi postoje, prikaz prazan	?
Proxy vraća `504`	?
Backend ima rezultat, preglednik javlja CORS	?

U paru: **hipoteza + dokaz koji je može opovrgnuti**.

Vježba

Demonstracija: isti put, četiri ishoda

Otvorite lokalnu aplikaciju i karticu **Network**.

1. Uspjeh: `200` i proizvodi
2. Prazno: `200` i `[]`
3. Kvar ovisnosti: `503`
4. Spor odgovor: čekanje pa rezultat

Predvidite → pokrenite → objasnite dokaz.

Vježba

Provjera: tri česte zablude

1. „HTTP je stateless pa aplikacija ne može pamtiti košaricu.”
2. „HTTPS znači da je korisnik ovlašten.”
3. „Server je vratio 200 pa se stranica ispravno prikazala.”

Za svaku tvrdnju ponudite **konkretan protuprimjer**.

Sažetak jednog zahtjeva

Pronađi odredište → pošalji → obradi → vrati → prikaži.

- Poruka ima namjeru, metapodatke i sadržaj.
- Stanje postoji na više mjesta.
- Veza, HTTP i sesija različiti su slojevi.
- Kašnjenje i kvar vežemo uz konkretnu granicu.

Početni spinner nije bio dovoljna dijagnoza.

Vježba

Osmislite sljedeći korak

Dodajemo `POST /orders`.

Zahtjev treba provjeriti ulaz, pronaći cijene, provjeriti zalihu i spremiti narudžbu.

Kako biste rasporedili odgovornosti?

Dodatni uvjet: baza kasni, a korisnik ponovno klikne **Naruči**.

Literatura: HTTP i preglednik

- RFC 9110: HTTP Semantics
- MDN: Overview of HTTP
- MDN: Using the Fetch API
- MDN: Using HTTP cookies
- MDN: HTTP request methods
- MDN: HTTP response status codes

Literatura: put kroz sustav

- MDN: DNS
- MDN: TCP
- MDN: Transport Layer Security
- MDN: Proxy servers and tunneling
- MDN: Connection management in HTTP/1.x
- MDN: Cross-Origin Resource Sharing
- Node.js: HTTP API